

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Set aesthetics for reports
sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (12, 6)

def professional_eda_pipeline(df):
    print("="*60)
    print("    PROFESSIONAL DATA SCIENCE EDA REPORT: RISK INSIGHTS")
    print("="*60)

    # 1. Structural Checklist
    print("\n[1] STRUCTURAL METRICS")
    print(f"Total Logged Line Items: {df.shape[0]}")
    print(f"Total Unique Invoices: {df['InvoiceNo'].nunique()}")
    print(f"Tracked Unique Stock Units: {df['StockCode'].nunique()}")
    print(f"Registered Customer Identities: {df['CustomerID'].nunique(dropna=True)}")

    # Missing Value Risk Audit
    print("\n[2] MISSING VALUE VULNERABILITY AUDIT")
    null_counts = df.isnull().sum()
    null_pct = (df.isnull().sum() / len(df)) * 100
    null_table = pd.concat([null_counts, null_pct], axis=1, keys=['Missing Count', 'Vulnerability Pct'])
    print(null_table[null_table['Missing Count'] > 0])

    # 2. Extract Implicit Target Risk States
    # Invoices starting with 'C' indicate cancellations/chargeback anomalies
    df['Is_Cancelled'] = df['InvoiceNo'].astype(str).str.startswith('C').astype(int)

    # Calculate Total Revenue per line item
    df['LineTotal'] = df['Quantity'] * df['UnitPrice']

    print("\n[3] RISK METRIC DISTRIBUTIONS")
    print(df[['Quantity', 'UnitPrice', 'LineTotal']].describe())

    # 3. Cancellation Risk Baseline Summary
    print("\n[4] TRANSACTION RISK BASELINES (CANCELLATIONS/CHARGEBACKS)")
    cancel_rate = df['Is_Cancelled'].mean() * 100
    print(f"Baseline Cancellation Rate: {cancel_rate:.2f}%")

    # Financial Exposure of Cancellations
    cancelled_rev = df[df['Is_Cancelled'] == 1]['LineTotal'].sum()
    orderly_rev = df[df['Is_Cancelled'] == 0]['LineTotal'].sum()
    print(f"Total Realized Financial Value: ${orderly_rev:,.2f}")
    print(f"Total Locked/Retracted Exposure Value: ${abs(cancelled_rev):,.2f}")

    # 4. Regional Exposure Stratification
    print("\n[5] COUNTRY-LEVEL RISK EXPOSURE PROFILE (TOP 10)")
    country_risk = df.groupby('Country').agg(
        Total_Items=('Quantity', lambda x: x.abs().sum()),
        Revenue_Stream=('LineTotal', lambda x: x[x > 0].sum()),
        Cancellation_Volume=('Is_Cancelled', 'sum'),
        Cancellation_Probability=('Is_Cancelled', 'mean')
    ).sort_values(by='Revenue_Stream', ascending=False).head(10)
    print(country_risk.to_string())

    # 5. Core Visualizations
    fig, axes = plt.subplots(2, 2, figsize=(16, 12))

    # Plot 1: Revenue vs Price Anomaly Scanner
    sns.scatterplot(data=df, x='UnitPrice', y='Quantity', hue='Is_Cancelled', alpha=0.7, ax=axes[0,0])
    axes[0,0].set_title("Outlier & Systemic Anomaly Detection Matrix")
    axes[0,0].set_xscale('log')

    # Plot 2: Line Item Value Volatility
    sns.histplot(df[df['LineTotal'] > 0]['LineTotal'], bins=50, kde=True, ax=axes[0,1], color='g')
    axes[0,1].set_title("Line-Item Financial Distribution (Log Scale)")
    axes[0,1].set_xscale('log')

    # Plot 3: Categorical Risk Profiles
    sns.barplot(data=country_risk.reset_index(), x='Cancellation_Probability', y='Country', ax=axes[1,0], palette="vlag")
    axes[1,0].set_title("Cancellation Probability Index by Region")

    # Plot 4: Missing Data Anomaly Map
    sns.heatmap(df.isnull(), cbar=False, cmap='viridis', yticklabels=False, ax=axes[1,1])
    axes[1,1].set_title("Structural Empty Vector Density Map (Missingness)")

    plt.tight_layout()

```

```
plt.show()
```

```
# To execute the EDA function on your runtime memory asset, uncomment below:
# professional_eda_pipeline(your_dataframe)
```

```
import io
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
from sklearn.pipeline import Pipeline

def build_and_evaluate_risk_model(df):
    print("\n" + "*60")
    print("  MACHINE LEARNING RISK ANALYTICS PIPELINE ENGINE")
    print("*60")

    # --- STEP 1: TARGET LABELLING & CLEANING ---
    df = df.copy()
    # Safely format data features
    df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])
    df['LineTotal'] = df['Quantity'] * df['UnitPrice']
    df['Is_Cancelled'] = df['InvoiceNo'].astype(str).str.startswith('C').astype(int)

    # Fill empty identifiers with an explicit anomaly tag flag string
    df['CustomerID'] = df['CustomerID'].fillna(-9999.0)

    # --- STEP 2: BEHAVIORAL PATTERN FEATURE ENGINEERING ---
    # Max date to establish historical recency baselines
    max_date = df['InvoiceDate'].max()

    # Aggregate transaction behavior at the Customer Level to predict systemic credit risk profiles
    customer_profile = df.groupby('CustomerID').agg(
        Recency_Days=('InvoiceDate', lambda x: (max_date - x.max()).days),
        Total_Transactions=('InvoiceNo', 'nunique'),
        Total_Units_Claimed=('Quantity', lambda x: x[x > 0].sum()),
        Total_Value_Spent=('LineTotal', lambda x: x[x > 0].sum()),
        Avg_Unit_Cost=('UnitPrice', 'mean'),
        Standard_Deviation_Cost=('UnitPrice', 'std'),
        Dominant_Country=('Country', 'first')
    ).fillna(0) # Catch any division errors/single row std anomalies

    # Establish target: Did this customer generate ANY cancelled invoices/chargeback risks?
    target_risk_labels = df.groupby('CustomerID')['Is_Cancelled'].max()

    # Merge engineered tensors
    ml_dataset = customer_profile.merge(target_risk_labels, left_index=True, right_index=True)

    X = ml_dataset.drop(columns=['Is_Cancelled'])
    y = ml_dataset['Is_Cancelled']

    # --- STEP 3: PIPELINE SCALING AND ENCODING TRANSFORMS ---
    numerical_features = ['Recency_Days', 'Total_Transactions', 'Total_Units_Claimed', 'Total_Value_Spent', 'Avg_Unit_Cost']
    categorical_features = ['Dominant_Country']

    preprocessor = ColumnTransformer(
        transformers=[
            ('num', StandardScaler(), numerical_features),
            ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
        ]
    )

    # --- STEP 4: BALANCED RANDOM FOREST PIPELINE ---
    model_pipeline = Pipeline(steps=[
        ('preprocessor', preprocessor),
        ('classifier', RandomForestClassifier(
            n_estimators=150,
            random_state=42,
            class_weight='balanced', # Crucial adjustment for handling minority risk targets
            max_depth=10
        ))
    ])

    # Stratified split to preserve class parity structures
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42, stratify=y)

    # --- STEP 5: PREDICTIVE FIT & PROFILING ---
    model_pipeline.fit(X_train, y_train)
    predictions = model_pipeline.predict(X_test)
    probabilities = model_pipeline.predict_proba(X_test)[: , 1]
```

```
# --- STEP 6: RISK METRICS SUMMARY ---
print("\n[1] PIPELINE PERFORMANCE MATRIX")
print(classification_report(y_test, predictions, target_names=['Low-Risk Account', 'High-Risk Account']))

print("[2] EXPOSURE ASSESSMENT AREA UNDER ROC (AUC-ROC)")
auc_score = roc_auc_score(y_test, probabilities)
print(f"Model AUC-ROC Score: {auc_score:.4f}")

print("\n[3] CONFUSION LOG MATRIX")
print(confusion_matrix(y_test, predictions))

return model_pipeline

# To initiate the machine learning training pipeline runtime block:
# pipeline_object = build_and_evaluate_risk_model(your_dataframe)
```

```
# Raw context parsing for isolated runtime execution verification
raw_data = """InvoiceNo,StockCode,Description,Quantity,InvoiceDate,UnitPrice,CustomerID,Country
536365,85123A,WHITE HANGING HEART T-LIGHT HOLDER,6,2010-12-01 08:26:00,2.55,17850.0,United Kingdom
536365,71053,WHITE METAL LANTERN,6,2010-12-01 08:26:00,3.39,17850.0,United Kingdom
C536379,D,Discount,-1,2010-12-01 09:41:00,27.5,14527.0,United Kingdom
536389,22941,CHRISTMAS LIGHTS 10 REINDEER,6,2010-12-01 10:03:00,8.5,12431.0,Australia
536544,21773,DECORATIVE ROSE BATHROOM BOTTLE,1,2010-12-01 14:32:00,2.51,,United Kingdom
C536757,84347,ROTATING SILVER ANGELS T-LIGHT HLDR,-9360,2010-12-02 14:23:00,0.03,15838.0,United Kingdom"""

# Structural Conversion
df = pd.read_csv(io.StringIO(raw_data))

# Execution Verification
professional_eda_pipeline(df)
trained_pipeline_engine = build_and_evaluate_risk_model(df)
```

